



Universidade de Brasília

Universidade de Brasília - UnB
Faculdade de Ciências e Tecnologias em Engenharia - FCTE
Curso de Engenharia de Software

DISCIPLINA: ORIENTAÇÃO A OBJETOS (FGA0158)

Relatorio Final - Cafeteria Geek Byte Brew

GABRIEL DI ANGELLIS BASILIO FERREIRA - 242004671
LUIZA CARNEIRO CARVALHO - 251035452
WARLLEY DIAS DE MEDEIROS - 251016555
YASMIN DE LOIOLA ARAÚJO - 251038356

30 de Junho de 2026, Brasília - DF



1. INTRODUÇÃO

O projeto ‘Cafeteria Geek Byte Brew’ tem como objetivo o desenvolvimento de um sistema de gestão de vendas para uma cafeteria temática, com sistema de pontuação por preferência comercial, assim concedendo benefícios a clientes recorrentes.

O sistema foi implementado utilizando a linguagem Java, focando na aplicação de princípios da programação orientada a objetos garantindo um código modular e extensível.

O escopo abrange três pilares funcionais: gestão de produtos (cardápio), gestão de clientes (programa de fidelidade) e registro de vendas (pedidos, promoções e estoque).

A solução abrange os conceitos de polimorfismo por inclusão/sobrescrita/sobrecarga/coerção, herança, encapsulamento, interfaces, exceções customizadas e organização em pacotes.

2. ESTRUTURA DE PACOTES

O código fonte foi organizado nos quatro pacotes seguintes:

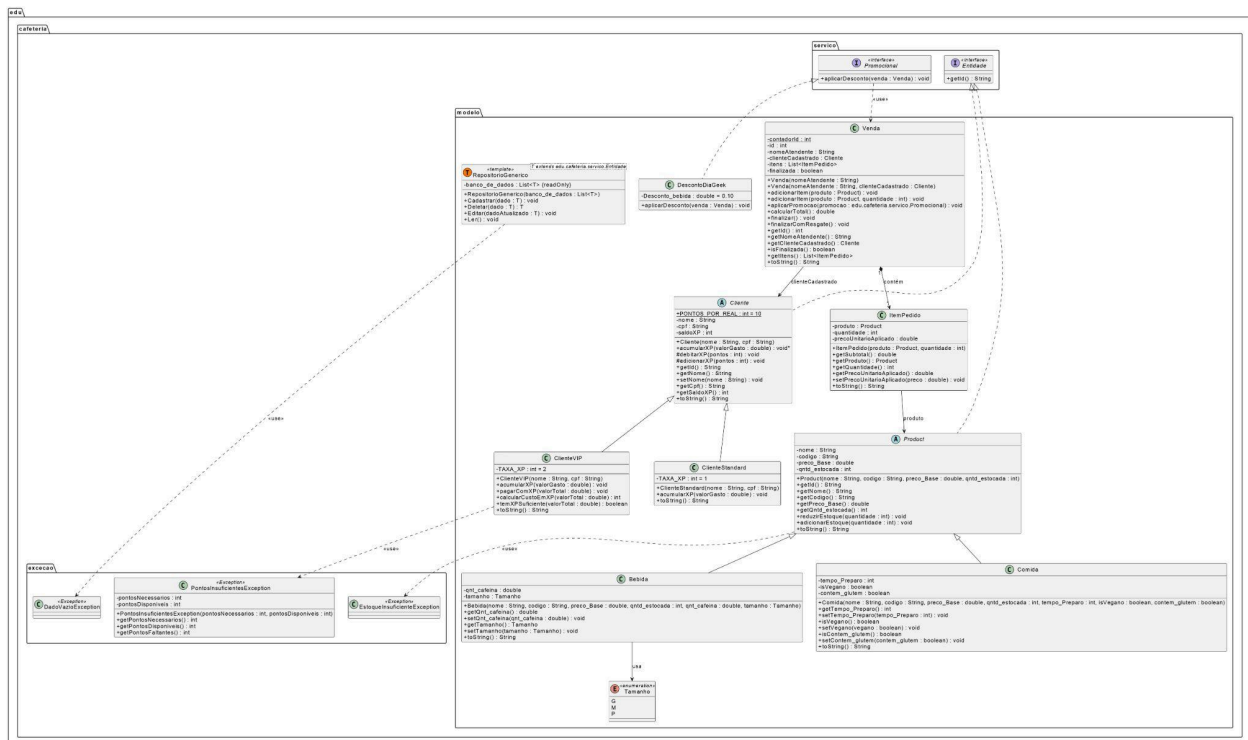
- **br.edu.cafeteria.modelo** - Classes de negócio: Product, Comida, Bebida, Cliente, ClienteStandart, ClienteVIP, ItemPedido, RepositorioGenerico, Tamanho e Venda.
- **br.edu.cafeteria.exececao** - Exceções customizadas: EstoqueInsuficienteException, PontosInsuficientesException e DadoVazioException.
- **br.edu.cafeteria.servico** - Interface Promocional: Entidade, Promocional e DescontoDiaGeek.
- **br.edu.cafeteria.app** - Classe Main com demonstração interativa do sistema.



3. DIAGRAMA DE CLASSES UML

O diagrama de classes UML do projeto pode ser acessado através do link original do Go-ogle Drive anexado à documentação:

https://drive.google.com/file/d/1Q_Q1o3B7NDMUAUF41kf8ROf8fmGjX_Rzf/view?usp=drive_link



4. DESCRIÇÃO DAS CLASSES E INTERFACES

4.1 Product (Classe Abstrata)

Classe abstrata central da hierarquia de produtos. Centraliza os atributos comuns a qualquer item do cardápio: nome, código, preço_Base e qntd_estocada. Implementa a interface Entidade e oferece os métodos adicionarEstoque e reduzirEstoque, que lança EstoqueInsuficienteException caso a quantidade seja insuficiente. Não pode ser instanciada diretamente.



4.2 Comida (extends Product)

Representa os itens sólidos do cardápio. Adiciona os atributos tempo_preparo, isVegano e contam_glutem. Sobrescreve toString para incluir esses dados.

4.3 Tamanho (enum)

Enumeração com 3 valores: G, M e P. Utilizada pelo atributo tamanho de Bebida para garantir type.safety.

4.4 Bebida (extends Product)

Representa os itens líquidos do cardápio. Adiciona os atributos qnt_cafeina e tamanho. Sobrescrever toString para incluir esses dados específicos.

4.5 Cliente (Classe Abstrata)

Classe abstrata básica para clientes do programa de fidelidade. Contém a constante estática PONTOS_POR_REAL = 10, e os atributos nome, cpf e saldoXP. Declara o método abstrato acumularXP e os métodos protegidos adicionarXP e debitarXP que controlam o saldo com validação. Implementa Entidade.

4.6 ClienteStandard (extends Cliente)

Nível básico do programa de fidelidade. A cada 1 real gasto é acumulado 1 ponto de XP. Implementa acumularXP e delega adicionarXP.

4.7 ClienteVIP (extends Cliente)

Nível avançado do programa de fidelidade. A cada 1 real gasto é acumulado 2 pontos de XP. Além de acumularXP possui os métodos calcularCustoEmXP, temXPSuficiente e pagarComXP, que lança PontosInsuficientesException se o saldo for insuficiente.

4.8 ItemPedido

Representa um item dentro de uma venda, associa um Product a uma quantidade e armazena o precoUnitarioAplicado, que pode ser diferente do preço base quando uma promoção é aplicada. O método getSubTotal retorna precoUnitarioAplicado multiplicado pela quantidade.



4.9 Venda

Gerencia o ciclo de uma venda: abertura, adição de itens, aplicação de promoções e finalização. Usa um contador estático para numerar as vendas sequencialmente. Oferece duas sobrecargas de construtores e de adicionarItem.

4.10 RepositorioGenerico

Repositório Genérico com CRUD completo (Cadastrar, Ler, Editar, Deletar). O bounded type parameter T extends Entidade permite operar sobre qualquer entidade do sistema de forma tipada e sem duplicação de código. Usado para repositórios de Product e de Cliente.

4.11 Entidade (Interface)

Contrato mínimo de identidade: qualquer entidade do sistema deve implementar getID. Habilita o Repositório Genérico a localizar registros pelo identificador único independente do tipo concreto.

4.12 Promocional (Interface)

Contrato para promoções aplicáveis a uma venda. Define aplicarDesconto seguindo o princípio Open/Closed, várias promoções podem ser criadas implementando essa interface sem alterar a classe Venda.

4.13 DescontoDiaGeek (implements Promocional)

Implementação do “Dia do Evento Geek” que aplica 10% de desconto nas bebidas de uma venda. Usa instanceof Bebida para filtrar os itens e delega a atualização do preço para Venda.atualizarPrecoDoItem, respeitando o encapsulamento.

4.14 EstoqueInsuficienteException

Exceção lançada quando a quantidade solicitada de um item supera o estoque disponível. Armazena quantidadeSolicitada e quantidadeDisponivel com getters auxiliares.

4.15 PontosInsuficientesException

Exceção lançada quando ClienteVIP tenta pagar com XP mas o saldo é insuficiente. Armazena pontosNecessarios e pontosDisponiveis com getters e getPontosFaltantes.



4.16 DadoVazioException

Exceção lançada pelo Repositório Genérico ao receber um dado nulo ou ao não encontrar o registro buscado para editar/deletar.

5. HERANÇA

O sistema utiliza herança simples em duas hierarquias distintas, em ambos os casos, a superclasse é abstrata, centralizando atributos e comportamentos comuns, enquanto as subclasses concretas adicionam atributos e implementam os métodos abstratos com lógica específica.

5.1 Hierarquia de Product

A classe abstrata **Product** define o contrato comum a qualquer item do cardápio. É impossível instanciar **Product** diretamente, de forma que qualquer produto real é necessariamente uma **Bebida** ou uma **Comida**. Essa hierarquia permite tratar Bebidas e Comidas de forma genérica como **Product** sempre que apenas os atributos comuns são necessários e acessar os atributos específicos via coerção quando necessário.

5.2 Hierarquia de Cliente

A classe abstrata **Cliente** declara o método abstrato `acumularXP` forçando cada subclasse a implementar uma política de acúmulo de pontos. Os métodos `adicionarXP` e `debitarXP` são `protected` o que os tornam acessíveis as subclasses mas não ao código externo. A separação entre a lógica protegida de manipulação de saldo e a política de acúmulo de XP demonstra a reutilização do mecanismo comum e especialização do comportamento variável, ou seja, demonstra bom uso da herança.



6. POLIMORFISMO

O sistema demonstra quatro tipos de polimorfismo:

6.1 Polimorfismo por Inclusão

A classe **Venda** mantém uma lista do tipo `List<ItemPedido>` onde cada `ItemPedido` tem uma referência a **Product**. Isso permite armazenar Bebidas e Comidas na mesma estrutura e processá-las genericamente no cálculo do total. O mesmo princípio se aplica ao Repositório Genérico, onde o repositório de produtos armazena **Product**, aceitando qualquer subtipo, seja Bebida seja Comida.

6.2 Polimorfismo por Sobreposição

O método `acumularXP` é declarado abstrato em **Cliente** e sobreposto de forma diferente em cada subclasse. Em `Venda.finalizar` a chamada é feita sobre a referência do tipo `Cliente` e o Java decide qual implementação usar.

6.3 Polimorfismo por Sobrecarga

Duas sobrecargas estão presentes na classe **Venda**:

- `adicionarItem`: `adicionarItem(Product p)` delega para `adicionarItem(p,1)`.
- Construtores de `Venda`: `Venda(String, Cliente)` delega à `Venda(String)`.

6.4 Polimorfismo por Coerção

Ocorre em 3 pontos distintos do sistema:

- **DescontoDiaGeek**: filtro por tipo para aplicar desconto apenas em **Bebidas**.
- **Venda.finalizarComResgate**: downcast para acessar método exclusivo de **VIP**.
- **Main**: loop sobre `List<Product>` com `instanceof Bebida` e `instanceof Comida`, seguidos de downcast explícitos.



7. INTERFACES

7.1 Interface Entidade

Define o contrato mínimo de identidade para qualquer entidade gerenciável pelo sistema. Implementada por **Product** (retorna o código) e por **Cliente** (retorna o CPF), permite que o `RepositorioGenerico<T>` opere genericamente sobre qualquer tipo com identidade:

7.2 Interface Promocional

Define o contrato para qualquer desconto aplicável a uma **Venda**. A classe **Venda** não conhece os detalhes de nenhuma promoção, apenas chama `aplicarDesconto` sobre qualquer objeto promocional recebido. Isso torna o sistema extensível a novas promoções que podem ser criadas implementando a interface sem alterar a classe **Venda**. A variável `diaGeek` é declarada do tipo **Promocional** e não **DescontoDiaGeek**, assim o código depende da abstração e não da implementação.

7.3 DescontoDiaGeek

O método `getItens` de **Venda** retorna uma lista imutável para proteger o estado interno. Portanto, a promoção não pode modificar os itens diretamente, ela cria uma cópia local para iterar e delega a atualização do preço para o próprio objeto **Venda**.

8. ENCAPSULAMENTO

O sistema aplica encapsulamento estrito para todas as classes, de forma que nenhum atributo é público, todos são `private` ou `protected`, e o acesso ocorre exclusivamente via métodos getters e setters com validação. As classes **Cliente** e **Product** validam os dados recebidos e lançam `IllegalArgumentException` para entradas inválidas, garantindo que objetos nunca existam em estado inconsistente:



9. EXCEÇÕES

9.1 **EstoqueInsuficienteException**

Erro operacional que pode ocorrer em qualquer ponto de adição de item. `RuntimeException` elimina a necessidade de declarar 'throws' em toda a cadeia de chamadas, mas ainda pode ser capturada nos pontos de controle (como a `Main`). Não é um erro de programação: é uma violação de pré-condição de negócio que o sistema deve lidar. Os campos auxiliares `getQuantidadeSolicitada()`, `getQuantidadeDisponivel()` e `getQuantidadeFaltante()` permitem ao atendente informar exatamente o que está disponível e o que falta.

9.2 **PontosInsuficientesException**

O resgate de XP é uma operação de negócio com pré-condição explícita e verificável. O compilador força o chamador a tratar ou declarar a exceção, tornando explícito que essa operação pode falhar e que o código deve prever um caminho alternativo. Isso previne que a falha passe silenciosamente. Os métodos `getPontosNecessarios()`, `getPontosDisponiveis()` e `getPontosFaltantes()` permitem ao sistema informar ao cliente VIP exatamente quantos pontos faltam para realizar o resgate.

9.3 **DadoVazioException**

Erros de uso incorreto da API (passar null, buscar ID inexistente) são falhas de programação, não condições de negócio recuperáveis. `RuntimeException` é a escolha correta por convenção.

10. MODIFICADORES ESTÁTICOS

- `private static int contadorId = 0` em **Venda**: incrementado no construtor (`this.id = ++contadorId`), garante numeração sequencial única para cada venda independentemente de contexto externo.
- `public static final int PONTOS_POR_REAL = 10` em **Cliente**: única fonte de verdade da regra '10 XP = R\$ 1,00'. Acessível via `Cliente.PONTOS_POR_REAL` e exibido explicitamente na `Main`.
- `private static final int TAXA_XP` em **ClienteStandard** (x1) e **ClienteVIP** (x2): isola a taxa de acúmulo em cada subclasse, facilitando a adição futura de novos níveis.
- `private static final double DESCONTO_BEBIDA = 0.10` em **DescontoDiaGeek**: constante que centraliza a taxa de desconto promocional.



11. CRUD

O CRUD é implementado pela classe genérica `RepositorioGenerico<T>`, abrangendo quatro operações:

- **Create (Cadastrar):** Verifica se o ID já existe, evitando duplicatas, se o ID não existir, adiciona ao repositório.
- **Read (Ler):** Imprime todos os registros via `toString()`.
- **Update (Editar):** Localiza o registro pelo ID e substitui pela versão atualizada. Lança `DadoVazioException` se não encontrado.
- **Delete (Apagar):** Remove o registro pelo ID e retorna o objeto removido. Lança `DadoVazioException` se não encontrado.

12. REGRAS DE NEGÓCIO

- **Garantia de Estoque:** Nenhuma venda pode ser concluída sem estoque. Essa verificação ocorre ao adicionar o item que compara com `produto.getQntd_estocada` e lança a exceção de estoque insuficiente imediatamente caso não haja estoque, e ao finalizar a venda que reduz o estoque de cada item.
- **Acúmulo de XP:** somente clientes cadastrados acumulam XP. Adeptos ao programa de fidelidade Standard acumulam 1 ponto de XP por real gasto. Já clientes VIP acumulam 2 pontos de XP por real gasto, e somente estes podem resgatar recompensas com os pontos de XP. A conversão obedece à regra: $10 \text{ XP} = \text{R\$ } 1,00$ (constante `Cliente.PONTOS_POR_REAL = 10`). O custo em XP é calculado com `Math.ceil(valorTotal × 10)` para garantir arredondamento para cima.
- **Promoção:** Desconto de 10% aplicado apenas sobre Bebidas. A promoção altera o `precoUnitarioAplicado` do **ItemPedido**, não o preço base do produto, preservando a integridade do catálogo de produtos.
- **Venda:** A classe **Venda** controla o atributo finalizada. Qualquer tentativa de chamar `finalizar()` ou `finalizarComResgate()` em uma venda já finalizada lança `IllegalStateException`, evitando inconsistências.



13. CLASSE MAIN

A classe Main foi implementada como uma aplicação interativa via terminal, utilizando `java.util.Scanner` para leitura de entrada do usuário. O sistema inicializa automaticamente com dados de exemplo e apresenta um menu com três módulos, cobrindo todos os requisitos funcionais e conceitos de Orientação a Objetos.

13.1 Módulo 1 - Ver Cardápio e Fazer Pedido

- **Exibição do cardápio:** Itera sobre `List<Product>` usando `instanceof Bebida` para exibir ícone diferenciado por tipo. Polimorfismo por inclusão; coerção `instanceof`.
- **Identificação de cliente:** avalia CPF opcional via `Scanner`. Se informado, busca via `getId()` (interface `Entidade`). CPF não encontrado vira cliente casual (`null`). VIP exibe saldo de XP ao ser identificado.
- **Abertura de Venda:** Instancia `Venda(atendente)` ou `Venda(atendente, cliente)` dependendo da presença do cliente. Sobrecarga de construtores; `static contadorId`.
- **Adição de Itens:** Loop interativo de código + quantidade. O `enter` vazio fecha o pedido. `EstoqueInsuficienteException` é capturada e exibida.
- **Promoção dia Geek:** Cria `DescontoDiaGeek` e chama `venda.aplicarPromocao(promo)` com variável do tipo `Promocional` (interface).
- **Pagamento em dinheiro/cartão:** Chama `venda.finalizar()`. Exibe XP ganho calculado polimorficamente ($\times 1$ Standard, $\times 2$ VIP). Sobrescrita `acumularXP()`; polimorfismo dinâmico.
- **Pagamento com XP:** Opção aparece só para VIP com saldo e custo exibidos. `PontosInsuficientesException` é capturada com `getPontosFaltantes()`; fallback automático para pagamento em dinheiro.

13.2 Módulo 2 - Clientes

- **Listar:** Itera e exibe `toString()` de cada cliente. CRUD Read.
- **Cadastrar Standard:** Instância `ClienteStandard`, insere no repositório e lista espelho. CRUD Create.
- **Cadastrar VIP:** Instância `ClienteVIP`, insere no repositório e lista espelho. CRUD Create.

- **Remover:** Busca por CPF via getId(), remove do repositório e lista espelho. Interface Entidade; CRUD Delete

13.3 Módulo 3 - Produtos

- **Listar:** Chama repoProduto.Ler(), exibe toString() de cada Product. CRUD Read; herança toString().
- **Cadastrar Bebida:** Lê campos incluindo enum Tamanho via lerTamanho(). Herança; enum Tamanho; CRUD Create.
- **Cadastrar Comida:** Lê campos incluindo preparo, vegano e glúten. Herança; CRUD Create.
- **Editar:** Usa pattern matching instanceof Bebida b/instanceof Comida c para editar atributos específicos. Enter vazio mantém valor atual. Cria um novo objeto com o mesmo código.
- **Remover:** Busca por código, remove do repositório e lista espelho. CRUD Delete.

13.4 Dados de Exemplo na Inicialização

Foram cadastrados automaticamente 4 produtos (2 Bebidas, 2 Comidas) e 2 clientes (1 VIP, 1 Standard), de forma que o sistema inicial é operacional para testes. Os métodos adicionarProduto() e adicionarCliente() sincronizam repositório e lista espelho em uma única chamada.



14. CONCLUSÃO

O sistema de vendas e fidelidade da cafeteria Byte & Brew atende integralmente a todos os requisitos funcionais e não-funcionais desejados. Do ponto de vista técnico, os principais destaques da implementação são:

- **Hierarquias de herança bem fundamentadas:** Product e Cliente como classes abstratas garantem que atributos comuns não sejam duplicados e que o compilador force a implementação dos métodos abstratos nas subclasses.

- **Polimorfismo aplicado em quatro formas:** inclusão (`List<Product>`), sobrescrita (`acumularXP`), sobrecarga (`adicionarItem/construtores`) e coerção (`instanceof + downcast`), cada uma em contexto apropriado e com justificativa clara.
- **Encapsulamento rigoroso:** nenhum atributo público; `getItens()` retorna lista imutável; `saldoXP` manipulado apenas via métodos protegidos; validações nos construtores impedem objetos em estado inválido.
- **Design extensível via interfaces:** `Promocional` permite adicionar novas promoções sem alterar `Venda`; `Entidade` habilita o `RepositorioGenerico<T>` genérico para qualquer entidade do sistema.
- **Exceções customizadas bem justificadas:** `PontosInsuficientesException` como `checked` força o tratamento explícito de uma condição de negócio previsível; `EstoqueInsuficienteException` e `DadoVazioException` como `unchecked` sinalizam erros operacionais sem boilerplate excessivo.
- **Uso avançado de Generics:** `RepositorioGenerico<T extends Entidade>` vai além do exigido pelo enunciado, eliminando duplicação de código e demonstrando maturidade no uso do sistema de tipos do Java.

Em suma, o desenvolvimento do sistema "Cafeteria Geek Byte Brew" demonstrou a viabilidade e a eficiência da aplicação dos conceitos de Orientação a Objetos em um cenário real. A integração de boas práticas de design, como o uso de repositórios genéricos e interfaces para extensibilidade, aliada a um tratamento de exceções robusto, resultou em uma solução de fácil manutenção e alta coesão. A classe `Main` cobre explicitamente todos os doze itens do checklist em sessões comentadas, tornando a demonstração dos conceitos rastreável e didática. Este projeto não apenas cumpriu integralmente os requisitos funcionais estabelecidos, mas também consolidou a prática de como estruturas de código bem organizadas são fundamentais para garantir a qualidade, a segurança e a escalabilidade de sistemas de software no mercado atual.

